

Multimodal Emotion Recognition

Training Notebook

-Speech Pipeline: Extracts Mel-frequency cepstral coefficients (MFCCs) from the audio signals and passes them through a Bidirectional LSTM (BiLSTM) with an attention mechanism for audio-based emotion classification.

-Text Pipeline: Leverages a pre-trained BERT bert-base-uncased architecture to process textual prosody descriptions and classify emotions based on linguistic features.

-Fusion Pipeline (Multimodal): Combines the speech features from the BiLSTM and text features from the BERT model using a gated fusion mechanism to perform joint, multimodal emotion recognition

STEP 1: Install Dependencies

```
!pip install -q torch torchaudio transformers librosa soundfile
scikit-learn matplotlib seaborn tqdm pandas numpy accelerate
!pip install -q kaggle
print('All packages installed')
```

All packages installed

STEP 2: Mount Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
import os
DRIVE_DIR = '/content/drive/MyDrive/EmotionRecognition'
os.makedirs(DRIVE_DIR, exist_ok=True)
os.makedirs(f'{DRIVE_DIR}/Results/plots', exist_ok=True)
print(f'Drive mounted. Saving to: {DRIVE_DIR}')
```

Mounted at /content/drive

Drive mounted. Saving to: /content/drive/MyDrive/EmotionRecognition

STEP 3: Download TESS Dataset

```
from google.colab import files
print('Upload your kaggle.json file:')
uploaded = files.upload()
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
```

```

!kaggle datasets download -d ejlok1/toronto-emotional-speech-set-tess
-p /content/data --unzip
print('Dataset downloaded!')
!ls /content/data/

Upload your kaggle.json file:

<IPython.core.display.HTML object>

Saving kaggle.json to kaggle.json
Dataset URL: https://www.kaggle.com/datasets/ejlok1/toronto-emotional-speech-set-tess
License(s): Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0)
Downloading toronto-emotional-speech-set-tess.zip to /content/data
100% 428M/428M [00:03<00:00, 130MB/s]

Dataset downloaded!
'tess toronto emotional speech set data'
'TESS Toronto emotional speech set data'

import os, glob
wav_files = glob.glob('/content/data/**/*.wav', recursive=True)
print(f'Found {len(wav_files)} WAV files')
if wav_files:
    DATA_ROOT = os.path.dirname(os.path.dirname(wav_files[0]))
    print(f'DATA_ROOT = {DATA_ROOT}')

Found 5600 WAV files
DATA_ROOT = /content/data/TESS Toronto emotional speech set data

```

STEP 4: Write Source Files

```

PROJECT_DIR = '/content/emotion_project'
os.makedirs(f'{PROJECT_DIR}/models/speech_pipeline', exist_ok=True)
os.makedirs(f'{PROJECT_DIR}/models/text_pipeline', exist_ok=True)
os.makedirs(f'{PROJECT_DIR}/models/fusion_pipeline', exist_ok=True)
os.makedirs(f'{PROJECT_DIR}/Results/plots', exist_ok=True)
for p in
['models', 'models/speech_pipeline', 'models/text_pipeline', 'models/
fusion_pipeline']:
    open(f'{PROJECT_DIR}/{p}/__init__.py', 'w').close()
print('Project structure created')

Project structure created

%%writefile /content/emotion_project/utils.py
import os
import numpy as np
import pandas as pd
import librosa

```

```

import torch
from pathlib import Path
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix, classification_report

EMOTION_LABELS = ['angry', 'disgust', 'fear', 'happy', 'neutral',
                  'ps', 'sad']
NUM_CLASSES = len(EMOTION_LABELS)
EMOTION_TO_IDX = {e: i for i, e in enumerate(EMOTION_LABELS)}
IDX_TO_EMOTION = {i: e for e, i in EMOTION_TO_IDX.items()}

def load_tess_dataset(data_root: str):
    records = []
    data_root = Path(data_root)
    for folder in sorted(data_root.iterdir()):
        if not folder.is_dir(): continue
        for wav_file in sorted(folder.glob('*.wav')):
            name = wav_file.stem.lower()
            parts = name.split('_')
            emotion = parts[-1]
            word = '_'.join(parts[1:-1])
            if emotion in ('pleasantsurprise', 'pleasantssurprise'):
emotion = 'ps'
            if emotion not in EMOTION_TO_IDX: continue
            records.append({'file_path': str(wav_file), 'emotion':
emotion,
                           'transcript': word, 'label':
EMOTION_TO_IDX[emotion]})
    df = pd.DataFrame(records)
    print(f'Loaded {len(df)} samples |
{df["emotion"].value_counts().to_dict()}')
    return df

def get_splits(df, test_size=0.15, val_size=0.15, seed=42):
    train_val, test = train_test_split(df, test_size=test_size,
                                       stratify=df['label'],
random_state=seed)
    rv = val_size / (1 - test_size)
    train, val = train_test_split(train_val, test_size=rv,
                                  stratify=train_val['label'],
random_state=seed)
    print(f'Split → train={len(train)} val={len(val)}
test={len(test)}')
    return train.reset_index(drop=True), val.reset_index(drop=True),
test.reset_index(drop=True)

def load_audio(path, sr=22050, duration=4.0):
    waveform, _ = librosa.load(path, sr=sr, mono=True)

```

```

max_len = int(sr * duration)
if len(waveform) < max_len:
    waveform = np.pad(waveform, (0, max_len - len(waveform)))
else:
    waveform = waveform[:max_len]
return waveform, sr

def extract_mfcc(waveform, sr=22050, n_mfcc=40, n_fft=512,
hop_length=256):
    mfcc = librosa.feature.mfcc(y=waveform, sr=sr, n_mfcc=n_mfcc,
                                n_fft=n_fft, hop_length=hop_length)

    delta = librosa.feature.delta(mfcc)
    delta2 = librosa.feature.delta(mfcc, order=2)
    return np.vstack([mfcc, delta, delta2]).T

def extract_prosody_cues(waveform, sr=22050):

    hop = 512

    # Energy
    rms = librosa.feature.rms(y=waveform, hop_length=hop)[0]
    energy_mean = float(rms.mean())
    energy_std = float(rms.std())

    f0 = librosa.yin(waveform, fmin=60, fmax=400, sr=sr,
hop_length=hop)
    voiced = f0 < 400 # yin marks unvoiced as fmax
    voiced_f0 = f0[voiced] if voiced.any() else np.array([0.0])
    pitch_mean = float(voiced_f0.mean())
    pitch_std = float(voiced_f0.std())
    pitch_range = float(voiced_f0.max() - voiced_f0.min())
    voiced_ratio = float(voiced.mean())

    zcr = librosa.feature.zero_crossing_rate(waveform, hop_length=hop)
[0]
    zcr_mean = float(zcr.mean())

    brightness_mean = float(
        librosa.feature.spectral_centroid(y=waveform, sr=sr,
hop_length=hop)[0].mean())

    harmonic, percussive = librosa.effects.hpss(waveform)
    hnr = float(np.sqrt((harmonic**2).mean() + 1e-8) /
                (np.sqrt((percussive**2).mean() + 1e-8) + 1e-8))

    return dict(energy_mean=energy_mean, energy_std=energy_std,
                pitch_mean=pitch_mean, pitch_std=pitch_std,
                pitch_range=pitch_range, voiced_ratio=voiced_ratio,
                zcr_mean=zcr_mean, brightness_mean=brightness_mean,

```

```

hnr=hnr)

def prosody_cues_to_text(cues: dict) -> str:
    parts = []
    if cues['energy_mean'] > 0.08: parts.append('very loud and energetic voice')
    elif cues['energy_mean'] > 0.04: parts.append('moderately loud voice')
    else: parts.append('quiet and soft voice')
    if cues['energy_std'] > 0.04: parts.append('with highly variable loudness')
    if cues['pitch_mean'] > 220: parts.append('high pitched tone')
    elif cues['pitch_mean'] > 150: parts.append('medium pitched tone')
    elif cues['pitch_mean'] > 0: parts.append('low pitched tone')
    if cues['pitch_range'] > 150: parts.append('wide pitch variation')
    elif cues['pitch_range'] > 60: parts.append('moderate pitch variation')
    else: parts.append('narrow pitch variation')
    if cues['pitch_std'] > 60: parts.append('unstable fluctuating pitch')
    if cues['voiced_ratio'] > 0.7: parts.append('highly voiced speech')
    elif cues['voiced_ratio'] < 0.3: parts.append('breathy unvoiced speech')
    if cues['zcr_mean'] > 0.08: parts.append('fast speaking rate')
    elif cues['zcr_mean'] < 0.04: parts.append('slow speaking rate')
    else: parts.append('normal speaking rate')
    if cues['brightness_mean'] > 3000: parts.append('bright sharp timbre')
    elif cues['brightness_mean'] < 1500: parts.append('dark warm timbre')
    if cues['hnr'] > 2.0: parts.append('clear tonal quality')
    elif cues['hnr'] < 0.8: parts.append('noisy rough voice quality')
    return 'The speaker has ' + ', '.join(parts) + '.'

def save_results(results_dir, model_name, y_true, y_pred, history=None):
    os.makedirs(results_dir, exist_ok=True)
    report = classification_report(y_true, y_pred, target_names=EMOTION_LABELS, output_dict=True)
    report_df = pd.DataFrame(report).transpose()
    report_df.to_csv(os.path.join(results_dir, f'{model_name}_accuracy_table.csv'))
    print(f'\n=== {model_name} ===')

```

```

    print(classification_report(y_true, y_pred,
target_names=EMOTION_LABELS))
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(9,7))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=EMOTION_LABELS,
yticklabels=EMOTION_LABELS)
    plt.title(f'{model_name} Confusion Matrix'); plt.tight_layout()
    plt.savefig(os.path.join(results_dir,
f'{model_name}_confusion_matrix.png'), dpi=150)
    plt.close()
    if history:
        fig, axes = plt.subplots(1, 2, figsize=(12,4))
        epochs = range(1, len(history['train_loss'])+1)
        axes[0].plot(epochs, history['train_loss'], label='Train')
        axes[0].plot(epochs, history['val_loss'], label='Val')
        axes[0].set_title('Loss'); axes[0].legend()
        axes[1].plot(epochs, history['train_acc'], label='Train')
        axes[1].plot(epochs, history['val_acc'], label='Val')
        axes[1].set_title('Accuracy'); axes[1].legend()
        plt.tight_layout()
        plt.savefig(os.path.join(results_dir,
f'{model_name}_history.png'), dpi=150)
        plt.close()
    return report_df

def set_seed(seed=42):
    import random; random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available(): torch.cuda.manual_seed_all(seed)

def get_device():
    d = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print(f'Using device: {d}'); return d

Writing /content/emotion_project/utils.py

```

STEP 4b: Pre-compute Prosody Descriptions

This replaces the per-sample audio processing that was slowing training to a crawl.

```

# -----
# PRE-COMPUTE all prosody descriptions
# Training datasets will load the CSV
# -----
import sys, os, glob
sys.path.insert(0, '/content/emotion_project')
import pandas as pd
from tqdm import tqdm

```

```

from utils import (
    load_tess_dataset, load_audio,
    extract_prosody_cues, prosody_cues_to_text
)

CACHE_CSV = '/content/emotion_project/prosody_descriptions.csv'

if os.path.exists(CACHE_CSV):
    print(f'Cache already exists: {CACHE_CSV} - skipping.')
else:
    df = load_tess_dataset(DATA_ROOT)
    descriptions = []
    print('Extracting prosody cues (runs once)...')
    for _, row in tqdm(df.iterrows(), total=len(df)):
        waveform, sr = load_audio(row['file_path'])
        cues = extract_prosody_cues(waveform, sr)
        descriptions.append(prosody_cues_to_text(cues))
    df['prosody_text'] = descriptions
    df.to_csv(CACHE_CSV, index=False)
    print(f'Saved {len(df)} descriptions → {CACHE_CSV}')

# Preview
import pandas as pd
df_check = pd.read_csv(CACHE_CSV)
print('\nSample descriptions:')
for i in range(3):
    print(f'    [{df_check.iloc[i]["emotion"]} {df_check.iloc[i]
["prosody_text"]}']

Loaded 2800 samples | {'fear': 400, 'ps': 400, 'sad': 400, 'angry':
400, 'disgust': 400, 'happy': 400, 'neutral': 400}
Extracting prosody cues (runs once)...

```

100%|██████████| 2800/2800 [08:11<00:00, 5.69it/s]

Saved 2800 descriptions →
/content/emotion_project/prosody_descriptions.csv

Sample descriptions:

[fear] The speaker has quiet and soft voice, high pitched tone, wide pitch variation, unstable fluctuating pitch, normal speaking rate, dark warm timbre.

[fear] The speaker has quiet and soft voice, high pitched tone, wide pitch variation, normal speaking rate, dark warm timbre.

[fear] The speaker has quiet and soft voice, high pitched tone, wide pitch variation, unstable fluctuating pitch, normal speaking rate, dark warm timbre.

```

%%writefile /content/emotion_project/models/speech_pipeline/train.py
import os, sys
import numpy as np

```

```

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from tqdm import tqdm
sys.path.append('/content/emotion_project')
from utils import load_tess_dataset, get_splits, load_audio,
extract_mfcc, save_results, set_seed, get_device, NUM_CLASSES

class SpeechDataset(Dataset):
    def __init__(self, df, sr=22050, duration=4.0, max_len=345):
        self.df=df; self.sr=sr; self.duration=duration;
self.max_len=max_len
    def __len__(self): return len(self.df)
    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        waveform, sr = load_audio(row['file_path'], sr=self.sr,
duration=self.duration)
        features = extract_mfcc(waveform, sr=sr)
        T = features.shape[0]
        if T < self.max_len:
            features = np.vstack([features, np.zeros((self.max_len-T,
features.shape[1]), dtype=np.float32)])
        else:
            features = features[:self.max_len]
            mean = features.mean(0, keepdims=True); std = features.std(0,
keepdims=True)+1e-8
            return torch.tensor((features-mean)/std, dtype=torch.float32),
torch.tensor(row['label'], dtype=torch.long)

class SpeechEmotionModel(nn.Module):
    def __init__(self, input_size=120, hidden_size=256, num_layers=2,
num_classes=NUM_CLASSES, dropout=0.3):
        super().__init__()
        self.bilstm = nn.LSTM(input_size, hidden_size, num_layers,
batch_first=True,
                                bidirectional=True, dropout=dropout if
num_layers>1 else 0.0)
        self.attention = nn.Linear(hidden_size*2, 1)
        self.classifier = nn.Sequential(nn.Linear(hidden_size*2,256),
nn.ReLU(),
                                nn.Dropout(dropout),
nn.Linear(256, num_classes))
    def forward(self, x):
        out, _ = self.bilstm(x)
        w = torch.softmax(self.attention(out), dim=1)
        ctx = (w*out).sum(dim=1)
        return self.classifier(ctx)
    def get_representation(self, x):
        out, _ = self.bilstm(x)

```



```

        w = torch.softmax(self.attention(out), dim=1)
        return (w*out).sum(dim=1)

def train_epoch(model, loader, opt, crit, device):
    model.train(); loss_sum=correct=total=0
    for x, y in tqdm(loader, desc='train', leave=False):
        x,y = x.to(device), y.to(device); opt.zero_grad()
        logits=model(x); loss=crit(logits,y); loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(),1.0); opt.step()
        loss_sum+=loss.item()*y.size(0);
    correct+=(logits.argmax(1)==y).sum().item(); total+=y.size(0)
    return loss_sum/total, correct/total

@torch.no_grad()
def evaluate(model, loader, crit, device):
    model.eval(); loss_sum=correct=total=0; preds=[]; labels=[]
    for x, y in loader:
        x,y=x.to(device),y.to(device); logits=model(x);
    loss=crit(logits,y)
        loss_sum+=loss.item()*y.size(0); p=logits.argmax(1)
        correct+=(p==y).sum().item(); total+=y.size(0)
        preds.extend(p.cpu().tolist());
    labels.extend(y.cpu().tolist())
    return loss_sum/total, correct/total, preds, labels

def train(data_root, epochs=30, batch_size=32, lr=1e-3,
results_dir='Results'):
    set_seed(42); device=get_device()
    df=load_tess_dataset(data_root); tr,va,te=get_splits(df)

    trl=DataLoader(SpeechDataset(tr),batch_size,shuffle=True,num_workers=2
, pin_memory=True)

    vl=DataLoader(SpeechDataset(va),batch_size,shuffle=False,num_workers=2
)

    tel=DataLoader(SpeechDataset(te),batch_size,shuffle=False,num_workers=
2)

    model=SpeechEmotionModel().to(device)
    crit=nn.CrossEntropyLoss()
    opt=torch.optim.AdamW(model.parameters(),lr=lr,weight_decay=1e-4)

    sched=torch.optim.lr_scheduler.ReduceLROnPlateau(opt,mode='max',patien
ce=5,factor=0.5)
    hist={'train_loss':[],'val_loss':[],'train_acc':[],'val_acc':[]}
    best=0.0; spath=os.path.join(results_dir,'speech_best_model.pt')
    print('\n=== SPEECH TRAINING ===')
    for ep in range(1,epochs+1):
        tl,ta=train_epoch(model,trl,opt,crit,device)
        vl_,va_,_,_=evaluate(model,vl,crit,device); sched.step(va_)

```

```

        hist['train_loss'].append(tl); hist['val_loss'].append(vl_)
        hist['train_acc'].append(ta); hist['val_acc'].append(va_)
        print(f'Ep {ep:03d}/{epochs} | TrLoss {tl:.4f} TrAcc {ta:.4f}
| VaLoss {vl:.4f} VaAcc {va:.4f}')
        if va_>best: best=va_; torch.save(model.state_dict(),spath);
print(f' Saved best ({best:.4f})')
        model.load_state_dict(torch.load(spath,map_location=device))
        _,acc,yp,yt=evaluate(model,tel,crit,device)
        print(f'Test Accuracy: {acc:.4f}')
        save_results(results_dir,'speech',yt,yp,hist)
        return model

```

Writing /content/emotion_project/models/speech_pipeline/train.py

```
%%writefile /content/emotion_project/models/text_pipeline/train.py
```

```

# -----
# Text Pipeline
# -----
import os, sys
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from transformers import BertTokenizer, BertModel
from tqdm import tqdm
sys.path.append('/content/emotion_project')
from utils import load_tess_dataset, get_splits, save_results,
set_seed, get_device, NUM_CLASSES

CACHE_CSV = '/content/emotion_project/prosody_descriptions.csv'

class TextDataset(Dataset):
    """Reads prosody_text from the pre-computed cache CSV.
    No audio loading, no feature extraction – pure text tokenisation.
    """
    def __init__(self, df, tokenizer, max_len=64):
        # df must have columns: file_path, label, prosody_text
        self.df = df.reset_index(drop=True)
        self.tok = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        text = f"Spoken word: {row['transcript']}. Voice cues:
{row['prosody_text']}"
        enc = self.tok(

```

```

        text,
        max_length=self.max_len,
        padding='max_length',
        truncation=True,
        return_tensors='pt'
    )
    return {
        'input_ids': enc['input_ids'].squeeze(0),
        'attention_mask': enc['attention_mask'].squeeze(0),
        'token_type_ids': enc['token_type_ids'].squeeze(0),
        'label': torch.tensor(row['label'], dtype=torch.long)
    }

class TextEmotionModel(nn.Module):
    def __init__(self, bert_name='bert-base-uncased',
                  num_classes=NUM_CLASSES, dropout=0.3,
                  unfreeze_last_n=6):
        super().__init__()
        self.bert = BertModel.from_pretrained(bert_name)
        for p in self.bert.parameters(): p.requires_grad = False
        total = len(self.bert.encoder.layer)
        for i in range(total - unfreeze_last_n, total):
            for p in self.bert.encoder.layer[i].parameters():
                p.requires_grad = True
        for p in self.bert.pooler.parameters(): p.requires_grad = True
        self.classifier = nn.Sequential(
            nn.Dropout(dropout), nn.Linear(768, 256),
            nn.ReLU(), nn.Dropout(dropout), nn.Linear(256,
num_classes)
        )

        def forward(self, input_ids, attention_mask, token_type_ids=None):
            out = self.bert(input_ids=input_ids,
attention_mask=attention_mask,
                        token_type_ids=token_type_ids)
            return self.classifier(out.pooler_output)

        def get_representation(self, input_ids, attention_mask,
token_type_ids=None):
            out = self.bert(input_ids=input_ids,
attention_mask=attention_mask,
                        token_type_ids=token_type_ids)
            return out.pooler_output

    def train_epoch(model, loader, opt, crit, device):
        model.train(); loss_sum=correct=total=0
        for b in tqdm(loader, desc='train', leave=False):
            ids = b['input_ids'].to(device)

```

```

        attn = b['attention_mask'].to(device)
        ttype = b['token_type_ids'].to(device)
        y = b['label'].to(device)
        opt.zero_grad()
        logits = model(ids, attn, ttype)
        loss = crit(logits, y)
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step()
        loss_sum += loss.item()*y.size(0)
        correct += (logits.argmax(1)==y).sum().item()
        total += y.size(0)
    return loss_sum/total, correct/total

@torch.no_grad()
def evaluate(model, loader, crit, device):
    model.eval(); loss_sum=correct=total=0; preds=[]; labels=[]
    for b in loader:
        ids = b['input_ids'].to(device)
        attn = b['attention_mask'].to(device)
        ttype = b['token_type_ids'].to(device)
        y = b['label'].to(device)
        logits = model(ids, attn, ttype)
        loss = crit(logits, y)
        loss_sum += loss.item()*y.size(0)
        p = logits.argmax(1)
        correct += (p==y).sum().item()
        total += y.size(0)
        preds.extend(p.cpu().tolist())
        labels.extend(y.cpu().tolist())
    return loss_sum/total, correct/total, preds, labels

def train(data_root, epochs=10, batch_size=32, lr=2e-5,
          results_dir='Results', bert_name='bert-base-uncased',
          cache_csv=CACHE_CSV):
    set_seed(42)
    device = get_device()
    tok = BertTokenizer.from_pretrained(bert_name)

    assert os.path.exists(cache_csv), \
        f'Cache CSV not found: {cache_csv} - run the pre-compute cell first!'
    df_cached = pd.read_csv(cache_csv)

    tr, va, te = get_splits(df_cached)
    trl = DataLoader(TextDataset(tr, tok), batch_size, shuffle=True,
num_workers=2, pin_memory=True)
    vl = DataLoader(TextDataset(va, tok), batch_size, shuffle=False,

```

```

num_workers=2)
    tel = DataLoader(TextDataset(te, tok), batch_size, shuffle=False,
num_workers=2)

    model = TextEmotionModel(bert_name).to(device)
    crit = nn.CrossEntropyLoss()
    opt = torch.optim.AdamW(
        filter(lambda p: p.requires_grad, model.parameters()),
        lr=lr, weight_decay=1e-4)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt,
T_max=epochs)
    hist = {'train_loss':[], 'val_loss':[], 'train_acc':[], 'val_acc':
[]}]
    best = 0.0
    spath = os.path.join(results_dir, 'text_best_model.pt')

    print('\n=== TEXT TRAINING (Prosody-Text → BERT) ===')
    for ep in range(1, epochs+1):
        tl, ta = train_epoch(model, trl, opt, crit, device)
        vl_, va_, _, _ = evaluate(model, vl, crit, device)
        sched.step()
        hist['train_loss'].append(tl); hist['val_loss'].append(vl_)
        hist['train_acc'].append(ta); hist['val_acc'].append(va_)
        print(f'Ep {ep:03d}/{epochs} | TrLoss {tl:.4f} TrAcc {ta:.4f}
| VaLoss {vl_:.4f} VaAcc {va_:.4f}')
        if va_ > best:
            best = va_
            torch.save(model.state_dict(), spath)
            print(f' Saved best ({best:.4f})')
    model.load_state_dict(torch.load(spath, map_location=device))
    _, acc, yp, yt = evaluate(model, tel, crit, device)
    print(f'Test Accuracy: {acc:.4f}')
    save_results(results_dir, 'text', yt, yp, hist)
    return model

```

Writing /content/emotion_project/models/text_pipeline/train.py

```

%%writefile /content/emotion_project/models/fusion_pipeline/train.py
# -----
# Fusion Pipeline
# Speech branch : audio → MFCC → BiLSTM
# Text branch   : reads pre-cached prosody_text → BERT
# -----
import os, sys
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from transformers import BertTokenizer

```

```

from tqdm import tqdm
sys.path.append('/content/emotion_project')
from utils import (
    load_tess_dataset, get_splits, load_audio, extract_mfcc,
    save_results, set_seed, get_device, NUM_CLASSES
)
from models.speech_pipeline.train import SpeechEmotionModel
from models.text_pipeline.train import TextEmotionModel

CACHE_CSV = '/content/emotion_project/prosody_descriptions.csv'

class MultimodalDataset(Dataset):
    """Speech: loads audio and computes MFCC.
    Text: reads pre-cached prosody description string.
    """
    def __init__(self, df, tokenizer,
                 sr=22050, duration=4.0,
                 max_len_audio=345, max_len_text=64):
        self.df = df.reset_index(drop=True)
        self.tok = tokenizer
        self.sr = sr; self.duration = duration
        self.mla = max_len_audio; self.mlt = max_len_text

    def __len__(self): return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]

        # — Speech branch: MFCC from audio —
        waveform, sr = load_audio(row['file_path'], sr=self.sr,
duration=self.duration)
        feat = extract_mfcc(waveform, sr=sr)
        T = feat.shape[0]
        if T < self.mla:
            feat = np.vstack([feat, np.zeros((self.mla-T,
feat.shape[1]), dtype=np.float32)])
        else:
            feat = feat[:self.mla]
            mean = feat.mean(0, keepdims=True); std = feat.std(0,
keepdims=True) + 1e-8
            feat = (feat - mean) / std

        # — Text branch: cached prosody description —
        text = str(row['prosody_text'])
        enc = self.tok(
            text, max_length=self.mlt,
            padding='max_length', truncation=True, return_tensors='pt'
        )
        return {

```

```

        'speech':          torch.tensor(feats, dtype=torch.float32),
        'input_ids':       enc['input_ids'].squeeze(0),
        'attention_mask':  enc['attention_mask'].squeeze(0),
        'token_type_ids':  enc['token_type_ids'].squeeze(0),
        'label':           torch.tensor(row['label'],
dtype=torch.long)
    }

class GatedFusion(nn.Module):
    def __init__(self, sd=512, td=768, fd=512):
        super().__init__()
        self.ps = nn.Linear(sd, fd); self.pt = nn.Linear(td, fd)
        self.gate = nn.Linear(sd+td, fd)
    def forward(self, s, t):
        sp = torch.tanh(self.ps(s)); tp = torch.tanh(self.pt(t))
        g = torch.sigmoid(self.gate(torch.cat([s, t], dim=1)))
        return g*sp + (1-g)*tp

class FusionEmotionModel(nn.Module):
    def __init__(self, num_classes=NUM_CLASSES, dropout=0.3):
        super().__init__()
        self.speech_enc = SpeechEmotionModel()
        self.text_enc = TextEmotionModel()
        self.speech_enc.classifier = nn.Identity()
        self.text_enc.classifier = nn.Identity()
        self.fusion = GatedFusion()
        self.classifier = nn.Sequential(
            nn.Linear(512, 256), nn.ReLU(),
            nn.Dropout(dropout), nn.Linear(256, num_classes)
        )
    def forward(self, speech, input_ids, attention_mask,
token_type_ids=None):
        s = self.speech_enc.get_representation(speech)
        t = self.text_enc.get_representation(input_ids,
attention_mask, token_type_ids)
        return self.classifier(self.fusion(s, t))
    def get_representation(self, speech, input_ids, attention_mask,
token_type_ids=None):
        s = self.speech_enc.get_representation(speech)
        t = self.text_enc.get_representation(input_ids,
attention_mask, token_type_ids)
        return self.fusion(s, t)

def train_epoch(model, loader, opt, crit, device):
    model.train(); loss_sum=correct=total=0
    for b in tqdm(loader, desc='train', leave=False):
        sp = b['speech'].to(device)

```

```

        ids    = b['input_ids'].to(device)
        attn   = b['attention_mask'].to(device)
        ttype  = b['token_type_ids'].to(device)
        y      = b['label'].to(device)
        opt.zero_grad()
        logits = model(sp, ids, attn, ttype)
        loss    = crit(logits, y)
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        opt.step()
        loss_sum += loss.item()*y.size(0)
        correct  += (logits.argmax(1)==y).sum().item()
        total    += y.size(0)
    return loss_sum/total, correct/total

```

```
@torch.no_grad()
```

```

def evaluate(model, loader, crit, device):
    model.eval(); loss_sum=correct=total=0; preds=[]; labels=[]
    for b in loader:
        sp    = b['speech'].to(device)
        ids   = b['input_ids'].to(device)
        attn  = b['attention_mask'].to(device)
        ttype = b['token_type_ids'].to(device)
        y     = b['label'].to(device)
        logits = model(sp, ids, attn, ttype)
        loss    = crit(logits, y)
        loss_sum += loss.item()*y.size(0)
        p = logits.argmax(1)
        correct += (p==y).sum().item()
        total   += y.size(0)
        preds.extend(p.cpu().tolist())
        labels.extend(y.cpu().tolist())
    return loss_sum/total, correct/total, preds, labels

```

```

def train(data_root,
          speech_ckpt='Results/speech_best_model.pt',
          text_ckpt='Results/text_best_model.pt',
          epochs=20, batch_size=16, lr=5e-5,
          results_dir='Results', cache_csv=CACHE_CSV):
    set_seed(42)
    device = get_device()
    tok    = BertTokenizer.from_pretrained('bert-base-uncased')

    assert os.path.exists(cache_csv), \
        f'Cache not found: {cache_csv} - run pre-compute cell first!'
    df = pd.read_csv(cache_csv)

    tr, va, te = get_splits(df)

```



```

    trl = DataLoader(MultimodalDataset(tr, tok), batch_size,
shuffle=True, num_workers=2, pin_memory=True)
    vl = DataLoader(MultimodalDataset(va, tok), batch_size,
shuffle=False, num_workers=2)
    tel = DataLoader(MultimodalDataset(te, tok), batch_size,
shuffle=False, num_workers=2)

    model = FusionEmotionModel().to(device)
    if os.path.exists(speech_ckpt):
        st = {k:v for k,v in torch.load(speech_ckpt,
map_location=device).items()
            if not k.startswith('classifier'))
        model.speech_enc.load_state_dict(st, strict=False)
        print(f'Loaded speech encoder from {speech_ckpt}')
    if os.path.exists(text_ckpt):
        st = {k:v for k,v in torch.load(text_ckpt,
map_location=device).items()
            if not k.startswith('classifier'))
        model.text_enc.load_state_dict(st, strict=False)
        print(f'Loaded BERT text encoder from {text_ckpt}')

    crit = nn.CrossEntropyLoss()
    bert_p = list(model.text_enc.bert.parameters())
    other_p = [p for p in model.parameters() if id(p) not in {id(bp)
for bp in bert_p}]
    opt = torch.optim.AdamW(
        [{'params': bert_p, 'lr': lr*0.1},
        {'params': other_p, 'lr': lr}],
        weight_decay=1e-4)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt,
T_max=epochs)
    hist = {'train_loss':[], 'val_loss':[], 'train_acc':[], 'val_acc':
[]}
    best = 0.0
    spath = os.path.join(results_dir, 'fusion_best_model.pt')

    print('\n=== FUSION TRAINING (BiLSTM + Prosody-Text BERT) ===')
    for ep in range(1, epochs+1):
        tl, ta = train_epoch(model, trl, opt, crit, device)
        vl_, va_, _, _ = evaluate(model, vl, crit, device)
        sched.step()
        hist['train_loss'].append(tl); hist['val_loss'].append(vl_)
        hist['train_acc'].append(ta); hist['val_acc'].append(va_)
        print(f'Ep {ep:03d}/{epochs} | TrLoss {tl:.4f} TrAcc {ta:.4f}
| VaLoss {vl_:.4f} VaAcc {va_:.4f}')
        if va_ > best:
            best = va_
            torch.save(model.state_dict(), spath)
            print(f' Saved best ({best:.4f})')
    model.load_state_dict(torch.load(spath, map_location=device))

```

```
_, acc, yp, yt = evaluate(model, tel, crit, device)
print(f'Test Accuracy: {acc:.4f}')
save_results(results_dir, 'fusion', yt, yp, hist)
return model
```

Writing /content/emotion_project/models/fusion_pipeline/train.py

STEP 5: Run All Training

```
import torch
print(f'GPU available: {torch.cuda.is_available()}')
if torch.cuda.is_available():
    print(f'GPU: {torch.cuda.get_device_name(0)}')
    print(f'VRAM: {torch.cuda.get_device_properties(0).total_memory /
1e9:.1f} GB')
```

GPU available: True

GPU: Tesla T4

VRAM: 15.6 GB

```
import sys, glob, os
sys.path.insert(0, '/content/emotion_project')
wavs = glob.glob('/content/data/**/*.*wav', recursive=True)
DATA_ROOT = str(sorted(set(os.path.dirname(os.path.dirname(w)) for w
in wavs))[0])
RESULTS_DIR = f'{DRIVE_DIR}/Results'
os.makedirs(RESULTS_DIR, exist_ok=True)
print(f'Data root: {DATA_ROOT}\nResults: {RESULTS_DIR}')
```

Data root: /content/data/TESS Toronto emotional speech set data

Results: /content/drive/MyDrive/EmotionRecognition/Results

— TRAIN SPEECH PIPELINE —

```
from models.speech_pipeline.train import train as speech_train
speech_model = speech_train(
    data_root=DATA_ROOT, epochs=30, batch_size=32,
    lr=1e-3, results_dir=RESULTS_DIR
)
```

Using device: cuda

Loaded 2800 samples | {'fear': 400, 'ps': 400, 'sad': 400, 'angry': 400, 'disgust': 400, 'happy': 400, 'neutral': 400}

Split → train=1960 val=420 test=420

=== SPEECH TRAINING ===

Ep 001/30 | TrLoss 0.9639 TrAcc 0.6281 | VaLoss 0.5429 VaAcc 0.8952
Saved best (0.8952)

Ep 002/30 | TrLoss 0.2745 TrAcc 0.9168 | VaLoss 0.0397 VaAcc 0.9905
Saved best (0.9905)

Ep 003/30 | TrLoss 0.0565 TrAcc 0.9842 | VaLoss 0.0404 VaAcc 0.9905

Ep 004/30 | TrLoss 0.0389 TrAcc 0.9929 | VaLoss 0.0291 VaAcc 0.9929
Saved best (0.9929)

Ep 005/30 | TrLoss 0.0191 TrAcc 0.9944 | VaLoss 0.0220 VaAcc 0.9929

Ep 006/30 | TrLoss 0.0125 TrAcc 0.9969 | VaLoss 0.0082 VaAcc 0.9976
Saved best (0.9976)

Ep 007/30 | TrLoss 0.0146 TrAcc 0.9964 | VaLoss 0.0206 VaAcc 0.9905

Ep 008/30 | TrLoss 0.0085 TrAcc 0.9969 | VaLoss 0.0342 VaAcc 0.9952

Ep 009/30 | TrLoss 0.0126 TrAcc 0.9974 | VaLoss 0.0075 VaAcc 0.9976

Ep 010/30 | TrLoss 0.0154 TrAcc 0.9959 | VaLoss 0.0281 VaAcc 0.9929

Ep 011/30 | TrLoss 0.0398 TrAcc 0.9883 | VaLoss 0.0077 VaAcc 0.9976

Ep 012/30 | TrLoss 0.0165 TrAcc 0.9944 | VaLoss 0.0222 VaAcc 0.9905

Ep 013/30 | TrLoss 0.0020 TrAcc 0.9985 | VaLoss 0.0152 VaAcc 0.9976

Ep 014/30 | TrLoss 0.0017 TrAcc 0.9995 | VaLoss 0.0168 VaAcc 0.9976

Ep 015/30 | TrLoss 0.0007 TrAcc 0.9995 | VaLoss 0.0219 VaAcc 0.9976

Ep 016/30 | TrLoss 0.0011 TrAcc 0.9995 | VaLoss 0.0160 VaAcc 0.9976

Ep 017/30 | TrLoss 0.0001 TrAcc 1.0000 | VaLoss 0.0139 VaAcc 0.9976

Ep 018/30 | TrLoss 0.0000 TrAcc 1.0000 | VaLoss 0.0138 VaAcc 0.9976

Ep 019/30 | TrLoss 0.0000 TrAcc 1.0000 | VaLoss 0.0134 VaAcc 0.9976

Ep 020/30 | TrLoss 0.0000 TrAcc 1.0000 | VaLoss 0.0129 VaAcc 0.9976

Ep 021/30 | TrLoss 0.0000 TrAcc 1.0000 | VaLoss 0.0123 VaAcc 0.9976

Ep 022/30 | TrLoss 0.0000 TrAcc 1.0000 | VaLoss 0.0112 VaAcc 0.9976

Ep 023/30 | TrLoss 0.0000 TrAcc 1.0000 | VaLoss 0.0109 VaAcc 0.9976

Ep 024/30 | TrLoss 0.0000 TrAcc 1.0000 | VaLoss 0.0089 VaAcc 0.9976

Ep 025/30 | TrLoss 0.0000 TrAcc 1.0000 | VaLoss 0.0032 VaAcc 0.9976

Ep 026/30 | TrLoss 0.0000 TrAcc 1.0000 | VaLoss 0.0032 VaAcc 0.9976

Ep 027/30 | TrLoss 0.0000 TrAcc 1.0000 | VaLoss 0.0032 VaAcc 0.9976

Ep 028/30 | TrLoss 0.0000 TrAcc 1.0000 | VaLoss 0.0032 VaAcc 0.9976

Ep 029/30 | TrLoss 0.0000 TrAcc 1.0000 | VaLoss 0.0032 VaAcc 0.9976

Ep 030/30 | TrLoss 0.0000 TrAcc 1.0000 | VaLoss 0.0032 VaAcc 0.9976
Test Accuracy: 0.9905

=== speech ===

	precision	recall	f1-score	support
angry	0.97	0.98	0.98	60
disgust	0.98	0.98	0.98	60
fear	1.00	0.97	0.98	60
happy	1.00	1.00	1.00	60
neutral	1.00	1.00	1.00	60
ps	0.98	1.00	0.99	60
sad	1.00	1.00	1.00	60
accuracy			0.99	420
macro avg	0.99	0.99	0.99	420
weighted avg	0.99	0.99	0.99	420

```
# — TRAIN TEXT PIPELINE
from models.text_pipeline.train import train as text_train
text_model = text_train(
    data_root=DATA_ROOT, epochs=15, batch_size=32,
    lr=2e-5, results_dir=RESULTS_DIR
)
```

Using device: cuda

```
{"model_id": "a5a2f915ad56475aaa6804ec8ab7fa08", "version_major": 2, "version_minor": 0}

{"model_id": "c8b5c7af9c6048b2a2e044767f29357e", "version_major": 2, "version_minor": 0}

{"model_id": "0e190d3278cc4c6fad76f7d39f50555d", "version_major": 2, "version_minor": 0}
```

Split → train=1960 val=420 test=420

```
{"model_id": "99c19190a5744ce4b3c92e80e76ed1f3", "version_major": 2, "version_minor": 0}

{"model_id": "24d96ac4b1704032b97918995f118dc4", "version_major": 2, "version_minor": 0}

{"model_id": "d206bb637a5d425d9fd4d4bdaa32234c", "version_major": 2, "version_minor": 0}
```

BertModel LOAD REPORT from: bert-base-uncased
Key | Status |

cls.seq_relationship.bias	UNEXPECTED		
cls.predictions.transform.LayerNorm.weight	UNEXPECTED		
cls.predictions.transform.LayerNorm.bias	UNEXPECTED		
cls.predictions.transform.dense.bias	UNEXPECTED		
cls.seq_relationship.weight	UNEXPECTED		
cls.predictions.transform.dense.weight	UNEXPECTED		
cls.predictions.bias	UNEXPECTED		

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

=== TEXT TRAINING (Prosody-Text → BERT) ===

Ep 001/15 | TrLoss 1.8096 TrAcc 0.2480 | VaLoss 1.6240 VaAcc 0.2857
Saved best (0.2857)

Ep 002/15 | TrLoss 1.5976 TrAcc 0.3148 | VaLoss 1.4809 VaAcc 0.4048
Saved best (0.4048)

Ep 003/15 | TrLoss 1.5168 TrAcc 0.3454 | VaLoss 1.4062 VaAcc 0.4119
Saved best (0.4119)

Ep 004/15 | TrLoss 1.3893 TrAcc 0.4474 | VaLoss 1.2336 VaAcc 0.5310
Saved best (0.5310)

Ep 005/15 | TrLoss 1.2509 TrAcc 0.5082 | VaLoss 1.1033 VaAcc 0.5738
Saved best (0.5738)

Ep 006/15 | TrLoss 1.1544 TrAcc 0.5485 | VaLoss 1.0368 VaAcc 0.6262
Saved best (0.6262)

Ep 007/15 | TrLoss 1.0831 TrAcc 0.5888 | VaLoss 1.0332 VaAcc 0.5905

Ep 008/15 | TrLoss 1.0542 TrAcc 0.6005 | VaLoss 1.0208 VaAcc 0.5643

Ep 009/15 | TrLoss 1.0203 TrAcc 0.6143 | VaLoss 0.9963 VaAcc 0.6000

Ep 010/15 | TrLoss 0.9950 TrAcc 0.6255 | VaLoss 0.9832 VaAcc 0.6238

Ep 011/15 | TrLoss 0.9736 TrAcc 0.6311 | VaLoss 0.9740 VaAcc 0.6381
Saved best (0.6381)

Ep 012/15 | TrLoss 0.9540 TrAcc 0.6296 | VaLoss 0.9694 VaAcc 0.6405
Saved best (0.6405)

Ep 013/15 | TrLoss 0.9575 TrAcc 0.6311 | VaLoss 0.9703 VaAcc 0.6024

Ep 014/15 | TrLoss 0.9499 TrAcc 0.6418 | VaLoss 0.9668 VaAcc 0.6357

Ep 015/15 | TrLoss 0.9277 TrAcc 0.6480 | VaLoss 0.9671 VaAcc 0.6357
Test Accuracy: 0.5952

=== text ===

	precision	recall	f1-score	support
angry	0.73	0.53	0.62	60
disgust	0.50	0.83	0.62	60
fear	0.51	0.55	0.53	60
happy	0.62	0.63	0.63	60
neutral	0.75	0.70	0.72	60
ps	0.42	0.18	0.26	60
sad	0.65	0.73	0.69	60
accuracy			0.60	420
macro avg	0.60	0.60	0.58	420
weighted avg	0.60	0.60	0.58	420

— TRAIN FUSION PIPELINE —

```
import importlib, models.fusion_pipeline.train as fusion_mod
importlib.reload(fusion_mod)
from models.fusion_pipeline.train import train as fusion_train
fusion_model = fusion_train(
    data_root=DATA_ROOT,
```

```

    speech_ckpt=os.path.join(RESULTS_DIR, 'speech_best_model.pt'),
    text_ckpt=os.path.join(RESULTS_DIR, 'text_best_model.pt'),
    epochs=20, batch_size=8, lr=2e-5,
    results_dir=RESULTS_DIR
)

```

Using device: cuda

Split → train=1960 val=420 test=420

```

{"model_id": "6ed9ac72d1c4441ea57b34f580c7c955", "version_major": 2, "version_minor": 0}

```

BertModel LOAD REPORT from: bert-base-uncased

Key	Status		
cls.seq_relationship.bias	UNEXPECTED		
cls.predictions.transform.LayerNorm.weight	UNEXPECTED		
cls.predictions.transform.LayerNorm.bias	UNEXPECTED		
cls.predictions.transform.dense.bias	UNEXPECTED		
cls.seq_relationship.weight	UNEXPECTED		
cls.predictions.transform.dense.weight	UNEXPECTED		
cls.predictions.bias	UNEXPECTED		

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

Loaded speech encoder from

/content/drive/MyDrive/EmotionRecognition/Results/speech_best_model.pt

Loaded BERT text encoder from

/content/drive/MyDrive/EmotionRecognition/Results/text_best_model.pt

=== FUSION TRAINING (BiLSTM + Prosody-Text BERT) ===

Ep 001/20 | TrLoss 1.2084 TrAcc 0.8556 | VaLoss 0.5734 VaAcc 0.9643
 Saved best (0.9643)

Ep 002/20 | TrLoss 0.2817 TrAcc 0.9755 | VaLoss 0.0957 VaAcc 0.9952
 Saved best (0.9952)

Ep 003/20 | TrLoss 0.0598 TrAcc 0.9949 | VaLoss 0.0359 VaAcc 0.9952

Ep 004/20 | TrLoss 0.0198 TrAcc 0.9990 | VaLoss 0.0230 VaAcc 0.9976
 Saved best (0.9976)

Ep 005/20 | TrLoss 0.0095 TrAcc 0.9995 | VaLoss 0.0194 VaAcc 0.9976

Ep 006/20 | TrLoss 0.0056 TrAcc 1.0000 | VaLoss 0.0173 VaAcc 0.9976

Ep 007/20 | TrLoss 0.0038 TrAcc 1.0000 | VaLoss 0.0164 VaAcc 0.9976

Ep 008/20 | TrLoss 0.0028 TrAcc 1.0000 | VaLoss 0.0160 VaAcc 0.9976

Ep 009/20 | TrLoss 0.0022 TrAcc 1.0000 | VaLoss 0.0157 VaAcc 0.9976

Ep 010/20 | TrLoss 0.0017 TrAcc 1.0000 | VaLoss 0.0155 VaAcc 0.9976

Ep 011/20 | TrLoss 0.0014 TrAcc 1.0000 | VaLoss 0.0155 VaAcc 0.9976

Ep 012/20 | TrLoss 0.0013 TrAcc 1.0000 | VaLoss 0.0156 VaAcc 0.9976

Ep 013/20 | TrLoss 0.0011 TrAcc 1.0000 | VaLoss 0.0155 VaAcc 0.9976

Ep 014/20 | TrLoss 0.0010 TrAcc 1.0000 | VaLoss 0.0156 VaAcc 0.9976

Ep 015/20 | TrLoss 0.0009 TrAcc 1.0000 | VaLoss 0.0156 VaAcc 0.9976

Ep 016/20 | TrLoss 0.0008 TrAcc 1.0000 | VaLoss 0.0156 VaAcc 0.9976

Ep 017/20 | TrLoss 0.0008 TrAcc 1.0000 | VaLoss 0.0156 VaAcc 0.9976

Ep 018/20 | TrLoss 0.0008 TrAcc 1.0000 | VaLoss 0.0156 VaAcc 0.9976

Ep 019/20 | TrLoss 0.0008 TrAcc 1.0000 | VaLoss 0.0156 VaAcc 0.9976

Ep 020/20 | TrLoss 0.0008 TrAcc 1.0000 | VaLoss 0.0156 VaAcc 0.9976
Test Accuracy: 0.9905

=== fusion ===

	precision	recall	f1-score	support
angry	1.00	0.97	0.98	60
disgust	0.98	0.98	0.98	60
fear	1.00	1.00	1.00	60
happy	1.00	1.00	1.00	60
neutral	1.00	0.98	0.99	60
ps	0.97	1.00	0.98	60
sad	0.98	1.00	0.99	60
accuracy			0.99	420
macro avg	0.99	0.99	0.99	420
weighted avg	0.99	0.99	0.99	420

STEP 6: View Results

```
import torch.nn as nn
from torch.utils.data import DataLoader
from transformers import BertTokenizer
import pandas as pd
from utils import get_splits, get_device
from models.fusion_pipeline.train import MultimodalDataset, evaluate
device = get_device()
tok = BertTokenizer.from_pretrained('bert-base-uncased')
import pandas as pd
df_cached =
pd.read_csv('/content/emotion_project/prosody_descriptions.csv')
_, _, te = get_splits(df_cached)
tel = DataLoader(MultimodalDataset(te, tok), batch_size=16,
shuffle=False, num_workers=2)
fusion_model.to(device).eval()
_, _, fusion_pred, fusion_true = evaluate(fusion_model, tel,
nn.CrossEntropyLoss(), device)
from sklearn.metrics import accuracy_score, classification_report
emotion_labels =
['angry', 'disgust', 'fear', 'happy', 'neutral', 'ps', 'sad']
print(f'Fusion Test Accuracy: {accuracy_score(fusion_true,
fusion_pred)*100:.2f}%')
print(classification_report(fusion_true, fusion_pred,
target_names=emotion_labels))
```

Using device: cuda
Split → train=1960 val=420 test=420
Fusion Test Accuracy: 99.05%

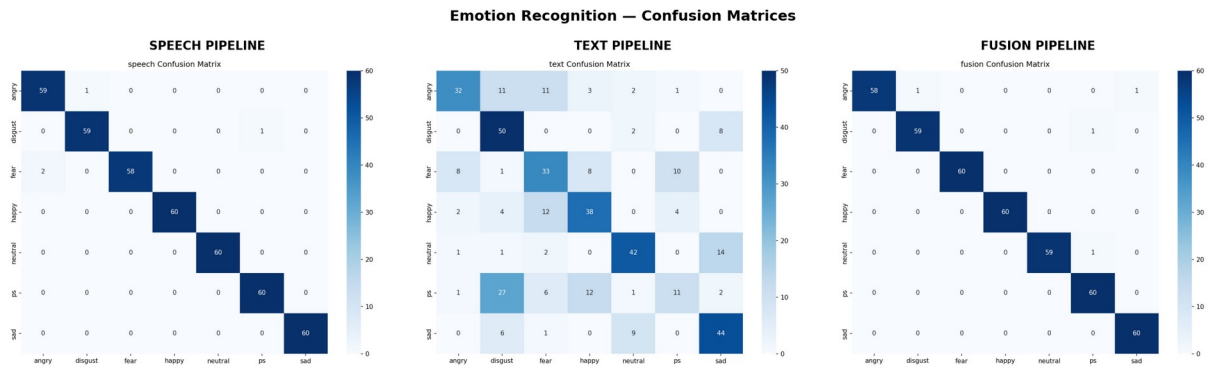
	precision	recall	f1-score	support
angry	1.00	0.97	0.98	60
disgust	0.98	0.98	0.98	60
fear	1.00	1.00	1.00	60
happy	1.00	1.00	1.00	60
neutral	1.00	0.98	0.99	60
ps	0.97	1.00	0.98	60
sad	0.98	1.00	0.99	60
accuracy			0.99	420
macro avg	0.99	0.99	0.99	420
weighted avg	0.99	0.99	0.99	420

```
import os, pandas as pd
print('\n' + '='*50 + '\nOVERALL ACCURACY SUMMARY\n' + '='*50)
for name in ['speech', 'text', 'fusion']:
    csv = os.path.join(RESULTS_DIR, f'{name}_accuracy_table.csv')
    if os.path.exists(csv):
        df2 = pd.read_csv(csv, index_col=0)
        acc = float(df2.loc['accuracy', 'f1-score'])
        print(f'{name.capitalize():10s}: {acc*100:.2f}%')
```

OVERALL ACCURACY SUMMARY

```
=====
Speech      : 99.05%
Text        : 59.52%
Fusion      : 99.05%
```

```
import matplotlib.pyplot as plt, matplotlib.image as mpimg
fig, axes = plt.subplots(1, 3, figsize=(22, 7))
for ax, name in zip(axes, ['speech', 'text', 'fusion']):
    p = os.path.join(RESULTS_DIR, f'{name}_confusion_matrix.png')
    if os.path.exists(p):
        ax.imshow(mpimg.imread(p)); ax.axis('off')
        ax.set_title(f'{name.upper()} PIPELINE', fontsize=15,
fontweight='bold')
plt.suptitle('Emotion Recognition – Confusion Matrices', fontsize=18,
fontweight='bold')
plt.tight_layout(); plt.show()
```



STEP 7: Save to Google Drive

```
import shutil
src_code_dest = f'{DRIVE_DIR}/project_code'
if os.path.exists(src_code_dest): shutil.rmtree(src_code_dest)
shutil.copytree('/content/emotion_project', src_code_dest)
print('Saved to Drive!')
for f in sorted(os.listdir(RESULTS_DIR)):
    fp = os.path.join(RESULTS_DIR, f)
    if os.path.isfile(fp): print(f' {f:<45}'
{os.path.getsize(fp)/1e6:.1f} MB')
```

Saved to Drive!

fusion_accuracy_table.csv	0.0 MB
fusion_best_model.pt	453.2 MB
fusion_confusion_matrix.png	0.1 MB
fusion_history.png	0.1 MB
speech_accuracy_table.csv	0.0 MB
speech_best_model.pt	9.9 MB
speech_confusion_matrix.png	0.1 MB
speech_history.png	0.1 MB
text_accuracy_table.csv	0.0 MB
text_best_model.pt	438.8 MB
text_confusion_matrix.png	0.1 MB
text_history.png	0.1 MB